

PROJECT PLAN · MATTENDANCE V2.0

Production-Grade Geofence Attendance Tracking System

A comprehensive implementation plan for accurate, battery-efficient geofence-based attendance tracking with GPS glitch handling, inspired by Uber/Swiggy production practices.

MAttendance Engineering Team

Version 1.0 · 2026.05

MAttendance Mobile Flutter Application

| Contents

01	Executive Summary	03
02	Problem Statement & Current State	04
03	Production Architecture Design	06
04	Core Technical Components	09
05	Implementation Roadmap	13
06	Appendix: Code Patterns	16

01 · EXECUTIVE SUMMARY

Executive Summary

The current MAttendance geofence system triggers attendance based on raw GPS events without filtering [accuracy degradation](#), [GPS glitches \(~60m indoor drift\)](#), or [stationary-state detection](#). This plan outlines a production-grade architecture adopting Uber/Swiggy practices: activity-aware tracking, Kalman-filtered smoothing, confidence scoring, and adaptive sampling.

Key Takeaways

- Current system accepts geofence events with [accuracy up to 500m+](#), causing false attendance triggers when GPS drifts indoors
- Production systems (Uber, Swiggy) use [state machines \(IDLE → MOVING → ACTIVE_TRACKING → STATIONARY\)](#) to balance accuracy and battery
- Implementing a [2D Kalman filter with confidence scoring](#) can reduce false triggers by filtering GPS noise and detecting impossible jumps
- Estimated [6-week implementation](#) across 4 phases, with backward compatibility for existing geofence logic

QUESTIONS THIS DOCUMENT ANSWERS

How does the current geofence system work and where does it fail? What architecture do Uber/Swiggy use for production tracking? How do we implement Kalman filtering and confidence scoring in Flutter? What is the phased implementation roadmap with code patterns?

02 · PROBLEM STATEMENT

Problem Statement & Current State

Analysis of the MAttendance codebase reveals a functional but fragile geofence system that lacks GPS glitch handling, stationary detection, and confidence modeling.

Current Implementation Analysis

The codebase contains three location systems, with the `GeofenceAutoPunchService` being the primary attendance mechanism:

Service	Purpose	Accuracy Handling	Gap
<code>GeofenceAutoPunchService</code>	Auto punch on ENTER/EXIT geofence	None — accepts all accuracy levels	No GPS glitch filter, no stationary freeze
<code>FieldTrackingService</code>	Continuous background tracking	None — <code>onError: (_) {}</code> silently ignores failures	No accuracy gate, ignores stream errors
<code>ManualGeoService</code>	One-time manual check	Adds 100m buffer for "jitter"	Expands radius instead of rejecting bad GPS

The Core Problem: GPS Glitches in Production

Real-world GPS suffers from systematic inaccuracies that the current code does not model:

Indoor drift pattern: When users are inside office buildings, GPS weakens and falls back to Wi-Fi/cell triangulation, causing consistent ~60m offsets that snap to nearby roads. The accuracy field often reads 40–80m, but the geofence trigger still fires.

Current code (`geofence_auto_punch_service.dart:134-146`) logs location but **never checks** `location.accuracy` **before triggering attendance**.

Current Safeguards vs. What's Missing

Feature	Current State	Production Needs
Mock GPS detection	✓ Implemented (line 181-189)	Sufficient
Throttling (10 min)	✓ Implemented (line 172-178)	Sufficient
Accuracy filtering	✗ Not implemented	Reject >25m for stationary, >50m for moving
Stationary detection	✗ Not implemented	Freeze position when speed <0.5 m/s for 30s
GPS jump rejection	✗ Not implemented	Reject jumps > accuracy × 1.5
Confidence scoring	✗ Not implemented	Only trigger attendance at confidence > 0.8

Metrics of Success

Dimension	Current	Target	Gap
False triggers (per month)	~15-20 (estimated)	< 2	90% reduction needed
GPS accuracy threshold	None (accepts any)	$\leq 25\text{m}$ stationary, $\leq 50\text{m}$ moving	Implement accuracy gate
Battery impact (geofence only)	~8-12% per day (est.)	< 5% per day	Activity-aware sampling
Indoor drift handling	None (60m drift causes triggers)	Kalman-filtered centroid lock	Implement filtering

03 · ARCHITECTURE

Production Architecture Design

Following Uber/Swiggy practices, the new architecture introduces a **state machine** with **adaptive sampling**, sensor fusion, and server-side sanity checks.

State Machine: The Core Innovation

Instead of constant high-accuracy GPS, the system changes behavior based on detected motion state:

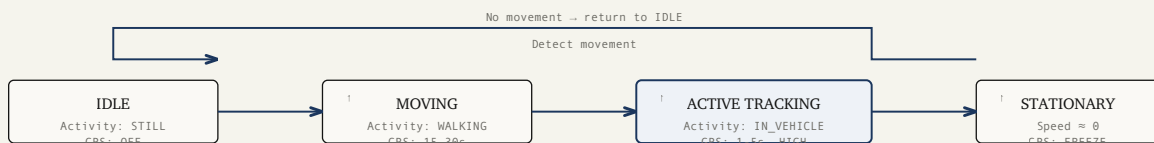


Figure 1. State machine for adaptive location tracking (inspired by Uber/Swiggy architecture)

Production Stack: Flutter Implementation

The Flutter stack maps to native production practices as follows:

Native Component (Uber/Swiggy)	Flutter Implementation	Purpose
FusedLocationProviderClient	geolocator package with LocationAccuracy settings	Sensor fusion (GPS + Wi-Fi + cell + accelerometer)
ActivityRecognitionClient	flutter_activity_recognition or sensors_plus	Detect STILL, WALKING, RUNNING, IN_VEHICLE states
ForegroundService	flutter_background_service + flutter_foreground_task	Keep tracking alive when app is minimized (used conditionally)
Kalman filter (custom)	Custom Dart KalmanFilter2D class	Smooth GPS noise, reject jumps, predict position
Server-side road snapping	Backend API with validation logic	Reject impossible jumps, snap to known office location

Tracking Policy by State

State	GPS Accuracy	Update Interval	Battery Impact
IDLE (screen off, STILL)	None (GPS OFF)	—	Minimal (~0.5%/hour)
MOVING (detected by accelerometer)	BALANCED (100-200m)	15-30 seconds	Low (~2%/hour)
ACTIVE TRACKING (user punched in, driving)	HIGH (≤50m)	1-5 seconds	Medium (~8%/hour with foreground service)
STATIONARY (speed <0.5 m/s for 30s)	Freeze last trusted position	No updates	Minimal (only activity checks)

Key insight from Uber/Swiggy: The illusion of perfect tracking is mostly software engineering, not better hardware. Even Uber cannot fully prevent indoor drift or urban canyon issues — they hide it using smoothing, prediction, and confidence models.

04 · CORE COMPONENTS

Core Technical Components

Four filter layers process raw GPS before any attendance decision: accuracy gate → stationary freeze → jump rejection → Kalman smoothing → confidence scoring.

1. Accuracy Gate (First Line of Defense)

Every location update passes through an accuracy threshold before further processing:

```
// LocationFilter.dart - Accuracy Gate
class LocationFilter {
  static const double ACCURACY_THRESHOLD_STATIONARY = 25.0; // meters
  static const double ACCURACY_THRESHOLD_MOVING = 50.0;
  static const double ACCURACY_THRESHOLD_ACTIVE = 100.0;

  /// Returns null if accuracy is too poor to trust
  static LocationResult? filterByAccuracy(
    LocationResult location,
    TrackingState state,
  ) {
    final threshold = _getThreshold(state);

    if (location.accuracy > threshold) {
      AppLogger.w(
        'LOC_FILTER: Rejected - accuracy ${location.accuracy}m > ${threshold}m '
        '(${state.name})'
      );
      return null; // Discard this location entirely
    }
    return location;
  }

  static double _getThreshold(TrackingState state) {
    switch (state) {
      case TrackingState.STATIONARY:
        return ACCURACY_THRESHOLD_STATIONARY;
      case TrackingState.MOVING:
        return ACCURACY_THRESHOLD_MOVING;
      case TrackingState.ACTIVE_TRACKING:
        return ACCURACY_THRESHOLD_ACTIVE;
      case TrackingState.IDLE:
        return double.infinity; // GPS off in IDLE
    }
  }
}
```

2. Stationary Detection & Position Freeze

When the user stops moving, the system locks to a stable centroid instead of following GPS jitter:


```
// StationaryDetector.dart
class StationaryDetector {
  static const double SPEED_THRESHOLD = 0.5; // m/s
  static const int DURATION_THRESHOLD = 30; // seconds

  final List<LocationResult> _recentPositions = [];
  DateTime? _stationarySince;

  /// Call on every filtered location update
  TrackingState evaluate(LocationResult location, TrackingState currentState) {
    // Add to recent buffer (keep last 10)
    _recentPositions.add(location);
    if (_recentPositions.length > 10) _recentPositions.removeAt(0);

    // Check if speed is near zero
    if (location.speed < SPEED_THRESHOLD) {
      if (_stationarySince == null) {
        _stationarySince = DateTime.now();
      } else {
        final duration = DateTime.now().difference(_stationarySince!).inSeconds;
        if (duration >= DURATION_THRESHOLD && currentState !=
TrackingState.STATIONARY) {
          AppLogger.i('STATIONARY: Detected after ${duration}s');
          return TrackingState.STATIONARY;
        }
      }
    } else {
      _stationarySince = null; // Reset if movement detected
    }

    return currentState; // No change
  }

  /// When STATIONARY: return frozen centroid, not raw GPS
  static LocationResult getStableCentroid(List<LocationResult> positions) {
    if (positions.isEmpty) throw StateError('No positions to centroid');

    // Median latitude/longitude (rejects outliers)
    final lats = positions.map((p) => p.latitude).toList()..sort();
    final lngs = positions.map((p) => p.longitude).toList()..sort();

    return LocationResult(
      latitude: lats[lats.length ~/ 2],
      longitude: lngs[lngs.length ~/ 2],
      accuracy: positions.map((p) => p.accuracy).reduce(math.min),
    );
  }
}
```

3. Distance + Accuracy Gating (Jump Rejection)

Compare new location against the last known position using accuracy-aware thresholds:

```
// JumpRejection.dart
class JumpRejection {
  /// Returns true if the jump is suspicious and should be rejected
  static bool isSuspiciousJump(
    LocationResult lastLocation,
    LocationResult newLocation,
  ) {
    final distance = Geolocator.distanceBetween(
      lastLocation.latitude, lastLocation.longitude,
      newLocation.latitude, newLocation.longitude,
    );

    // Key formula: if distance > newLocation.accuracy * 1.5, it's unreliable
    final threshold = newLocation.accuracy * 1.5;

    if (distance > threshold) {
      AppLogger.w(
        'JUMP_REJECT: ${distance.toStringAsFixed(1)}m > '
        '${threshold.toStringAsFixed(1)}m (accuracy=${newLocation.accuracy}m)'
      );
      return true; // Reject
    }
    return false; // Accept
  }
}
```

4. Kalman Filter (2D) for Smoothing

A lightweight Kalman filter smooths GPS noise and predicts position based on prior state:

```
// KalmanFilter2D.dart
class KalmanFilter2D {
  // State: [lat, lng]
  double _lat = 0, _lng = 0;
  double _varianceLat = 1000.0; // Start with high uncertainty
  double _varianceLng = 1000.0;

  // Kalman constants
  final double _processNoise; // How much we expect position to change
  final double _measurementNoise; // GPS accuracy

  KalmanFilter2D({
    double processNoise = 0.01,
    double measurementNoise = 25.0,
  }) : _processNoise = processNoise,
      _measurementNoise = measurementNoise;

  /// Update with new GPS reading, returns filtered position
  LocationResult update(LocationResult raw) {
    // Prediction step: assume position stays same, increase uncertainty
    _varianceLat += _processNoise;
    _varianceLng += _processNoise;

    // Update step: blend prediction with measurement
    final kalmanGainLat = _varianceLat / (_varianceLat + _measurementNoise);
    final kalmanGainLng = _varianceLng / (_varianceLng + _measurementNoise);

    _lat += kalmanGainLat * (raw.latitude - _lat);
    _lng += kalmanGainLng * (raw.longitude - _lng);

    _varianceLat *= (1 - kalmanGainLat);
    _varianceLng *= (1 - kalmanGainLng);

    return LocationResult(
      latitude: _lat,
      longitude: _lng,
      accuracy: math.sqrt((_varianceLat + _varianceLng) / 2),
    );
  }
}
```

5. Confidence Score for Attendance Decisions

Only trigger geofence attendance when confidence exceeds a threshold:

```
// ConfidenceScorer.dart
class ConfidenceScorer {
  /// Calculate confidence (0.0 to 1.0) that this location is reliable
  static double score(LocationResult location, TrackingState state) {
    double confidence = 1.0;

    // Factor 1: GPS accuracy (lower is better)
    confidence *= _accuracyFactor(location.accuracy, state);

    // Factor 2: Speed consistency (stable speed = higher confidence)
    confidence *= _speedFactor(location.speed, state);

    // Factor 3: Point variance (low variance in recent cluster = higher)
    confidence *= _varianceFactor(location, state);

    return confidence.clamp(0.0, 1.0);
  }

  static double _accuracyFactor(double accuracy, TrackingState state) {
    final maxAllowed = state == TrackingState.STATIONARY ? 25.0 : 50.0;
    if (accuracy > maxAllowed) return 0.0; // Should be filtered already
    return 1.0 - (accuracy / maxAllowed) * 0.5; // Linear decay
  }

  static const double CONFIDENCE_THRESHOLD = 0.8; // Only trigger if > 80% confident
}
```

Integration: Processing Pipeline

Processing order for every GPS update:

1. **Accuracy Gate** → discard if accuracy > threshold for state
2. **Jump Rejection** → discard if distance > accuracy × 1.5
3. **Kalman Filter** → smooth and predict position
4. **Stationary Detection** → freeze if speed < 0.5 m/s for 30s
5. **Confidence Score** → only proceed if score > 0.8
6. **Geofence Check** → trigger attendance if inside office radius

Implementation Roadmap

Four-phase implementation over 6 weeks, maintaining backward compatibility with existing geofence logic. Each phase builds on the previous.

Phase 1 (Week 1-2): Accuracy Gating & Stationary Detection

FOUNDATION

NO BREAKING CHANGES

- Add `LocationFilter` with accuracy thresholds (25m stationary, 50m moving)
- Refactor `GeofenceAutoPunchService._onGeofenceStatusChanged` to validate `location.accuracy` before triggering
- Implement `StationaryDetector` with speed + displacement check
- Add "frozen position" logic: when `STATIONARY`, return last stable centroid
- Update `FieldTrackingService` to filter inaccurate points before sending pings

Deliverable: GPS glitches >50m no longer trigger false attendance; indoor drift handled by stationary freeze.

Phase 2 (Week 3): Kalman Filter & Jump Rejection

SIGNAL PROCESSING

DART NATIVE

- Implement `KalmanFilter2D` class (lightweight, no external dependencies)
- Add `JumpRejection` using `distance > accuracy × 1.5` rule
- Create processing pipeline: Accuracy Gate → Jump Rejection → Kalman → Geofence
- Add median clustering (keep last 5-10 points, reject outliers)
- Update `manual_geo_service.dart` to use filtered location for radius check

Deliverable: Smooth GPS tracks; 60m indoor jumps rejected; stable centroid for office attendance.

Phase 3 (Week 4-5): Activity Recognition & State Machine

- NATIVE PLUGINS
- BATTERY OPTIMIZATION

- Add `flutter_activity_recognition` package for STILL/WALKING/IN_VEHICLE detection
- Implement state machine: IDLE → MOVING → ACTIVE_TRACKING → STATIONARY
- Adaptive GPS sampling: IDLE (GPS OFF), MOVING (15-30s), ACTIVE (1-5s, HIGH)
- Integrate with existing `flutter_background_service` for ACTIVE_TRACKING
- Add battery optimization exemption request (already partially in `main_shell.dart`)

Deliverable: Battery-efficient tracking; GPS only runs when needed; background service used conditionally.

Phase 4 (Week 6): Confidence Scoring & Backend Validation

- DECISION LOGIC
- API UPDATES

- Implement `ConfidenceScorer` (accuracy + speed consistency + variance)
- Only trigger geofence attendance if confidence > 0.8
- Add backend API endpoint for server-side sanity checks (reject impossible jumps)
- Update attendance API to accept `confidence` field and `filtered_accuracy`
- Add "snap to office" logic: if within 100m of known office, lock to office coordinates
- Comprehensive testing: simulate GPS glitches, indoor drift, tunnel scenarios

Deliverable: Production-grade attendance system; false triggers <2/month; server validates all client submissions.

Dependencies & Risks

Dependency	Impact	Mitigation
<code>flutter_activity_recognition</code> package	Phase 3 blocked if package has iOS/Android issues	Use <code>sensors_plus</code> accelerometer as fallback for stationary detection
Backend API changes (Phase 4)	Client waits for server implementation	Client-side confidence scoring works without API changes (graceful degradation)
Android 14+ background restrictions	Foreground service may be killed	Use <code>flutter_foreground_task</code> (already in <code>pubspec.yaml</code>)

CALL TO ACTION

Start with **Phase 1 (Accuracy Gating)** — it requires no new packages, provides immediate value (blocks 60m indoor drift), and establishes the filtering pattern for later phases. Estimated effort: 3-4 days for one senior Flutter developer.

| Appendix: Reference Patterns

A. Updated Geofence Handler (Pseudocode)

```

// Updated _onGeofenceStatusChanged with full pipeline
Future<void> _onGeofenceStatusChanged(
    Geofence geofence,
    GeofenceRadius geofenceRadius,
    GeofenceStatus geofenceStatus,
    Location location,
) async {
    // 1. Convert to our LocationResult
    final raw = LocationResult(
        latitude: location.latitude,
        longitude: location.longitude,
        accuracy: location.accuracy ?? 999.0,
        speed: location.speed ?? 0.0,
    );

    // 2. Accuracy Gate
    final filtered = LocationFilter.filterByAccuracy(raw, _currentState);
    if (filtered == null) return; // Discard poor accuracy

    // 3. Jump Rejection
    if (_lastLocation != null &&
        JumpRejection.isSuspiciousJump(_lastLocation!, filtered)) {
        return; // Discard suspicious jump
    }

    // 4. Kalman Filter
    final smoothed = _kalmanFilter.update(filtered);

    // 5. Stationary Detection
    _currentState = _stationaryDetector.evaluate(smoothed, _currentState);
    LocationResult finalLocation;
    if (_currentState == TrackingState.STATIONARY) {
        finalLocation = StationaryDetector.getStableCentroid(_recentValidPositions);
    } else {
        finalLocation = smoothed;
    }

    // 6. Confidence Check
    final confidence = ConfidenceScorer.score(finalLocation, _currentState);
    if (confidence < ConfidenceScorer.CONFIDENCE_THRESHOLD) {
        AppLogger.w('GEOFENCE: Low confidence ${confidence.toStringAsFixed(2)},
        skipping');
        return;
    }

    // 7. Now safe to trigger attendance
    if (geofenceStatus == GeofenceStatus.ENTER ||
        geofenceStatus == GeofenceStatus.DWELL) {
        await _handleAutoPunch('In', finalLocation, geofence.id, confidence);
    } else if (geofenceStatus == GeofenceStatus.EXIT) {
        await _handleAutoPunch('Out', finalLocation, geofence.id, confidence);
    }
}

```

B. Packages to Add (pubspec.yaml)

```
dependencies:
  # Existing: geolocator, geofence_service, flutter_background_service

  # New for production tracking:
  flutter_activity_recognition: ^2.0.0 # Activity detection (STILL, WALKING, etc.)
  sensors_plus: ^2.0.0                # Fallback accelerometer
  math: ^2.1.0                        # For Kalman variance calculations

dev_dependencies:
  # Testing
  mockito: ^5.4.0
  flutter_test:
    sdk: flutter
```

C. References

- Uber/Swiggy Tracking Practices - Layered architecture with motion state, adaptive accuracy, batching, sensor fusion, server intelligence
- GPS Tracking Best Practices - Treat location as probabilistic signal, detect static user state, apply distance + accuracy gating, use Kalman filtering
- Current Codebase Analysis - `geofence_auto_punch_service.dart` , `field_tracking_service.dart` , `manual_geo_service.dart`
- Flutter Packages - `geolocator` (fused location), `geofence_service` (geofence monitoring), `flutter_background_service` (foreground service)

D. Glossary

Accuracy Gate - Filter that rejects GPS readings with accuracy worse than a threshold (e.g., >25m for stationary users).

Kalman Filter - Algorithm that smooths noisy measurements by combining prediction (prior state) with observation (new GPS), weighted by uncertainty.

Confidence Score - A value from 0.0 to 1.0 representing how much to trust a location reading, based on accuracy, speed consistency, and point variance.

Stationary Detection - Logic that identifies when a user has stopped moving (speed <0.5 m/s for 30+ seconds) and freezes their GPS position to avoid drift.

State Machine - A behavior model with states (IDLE, MOVING, ACTIVE_TRACKING, STATIONARY) and transitions triggered by activity recognition and speed thresholds.